

Program Drill — Set 3 (harder)

Three programs where the structure is sound and the defects are in ordering, boundaries and failure paths

October 2026

How to run it. Same reading order. These are harder because the obvious shapes (`await` present, `try` present, `Map` used, inputs validated) are all there; the bugs are in what happens on the failure path, at the boundary, or between two correct lines. 40 minutes per program.

Program H — Rate-limited job runner with graceful shutdown

Requirement

1. `run(jobs)` executes jobs with at most `limit` in flight; returns results in input order.
2. On `SIGTERM`, stop starting new jobs, wait up to `graceMs` for running ones, then resolve with whatever finished and reject the rest with `ShutdownError`.
3. A job that throws is recorded as `{ ok: false, error }`; others as `{ ok: true, value }`; one failure never stops the others.
4. No timers or listeners remain after `run` resolves.

```
1  type Result<T> = { ok: true; value: T } | { ok: false; error: unknown };
2  class ShutdownError extends Error {}
3
4  export async function run<T>(jobs: Array<() => Promise<T>>, limit: number, graceMs: number):
Promise<Result<T>[]> {
5      const results: Result<T>[] = new Array(jobs.length);
6      let next = 0;
7      let shuttingDown = false;
8      const inflight = new Set<Promise<void>>();
9
10     const onTerm = () => { shuttingDown = true; };
11     process.on("SIGTERM", onTerm);
12
13     const startOne = (i: number) => {
14         const p = jobs[i]()
15         .then((value) => { results[i] = { ok: true, value }; })
16         .catch((error) => { results[i] = { ok: false, error }; })
17         .finally(() => { inflight.delete(p); });
18         inflight.add(p);
19     };
20
21     while (next < jobs.length && !shuttingDown) {
22         while (inflight.size < limit && next < jobs.length) startOne(next++);
23         await Promise.race(inflight);
24     }
25
26     if (shuttingDown) {
27         const timeout = new Promise<void>((r) => setTimeout(r, graceMs));
28         await Promise.race([Promise.all(inflight), timeout]);
29         for (let i = 0; i < jobs.length; i++) {
30             if (results[i] === undefined) results[i] = { ok: false, error: new ShutdownError() };
31         }
32     } else {
33         await Promise.all(inflight);
34     }
35
36     process.off("SIGTERM", onTerm);
37     return results;
38 }
```

Reading sequence

Step	Lines
types	1–2
state	5–8 (per call), 11 (process-level listener)
signature	4

Step	Lines
body	21–37
helpers	13–19 (<code>startOne</code>)
lifecycle	11, 27, 36
clauses	

Notes

Line	Note
5	<code>new Array(n)</code> is sparse: every slot is <code>undefined</code> until written; line 30 relies on that
11	listener added per call; removed at 36 only if <code>run</code> reaches 36
14–17	<code>p</code> is the chain; <code>.catch</code> records failures so the chain never rejects; <code>.finally</code> removes <code>p</code> from <code>inflight</code> ; <code>p</code> is referenced inside its own <code>finally</code> before being assigned: at the time the <code>finally</code> callback RUNS, <code>p</code> is assigned; fine
18	added after construction; the chain cannot settle before this line (microtasks run later); fine
22	fills up to <code>limit</code> ; <code>next++</code> increments after use
23	<code>Promise.race(inflight)</code> : waits for any one to finish; if <code>inflight</code> is EMPTY (<code>limit <= 0</code> , or <code>jobs</code> empty and loop entered), <code>Promise.race([])</code> never settles: hang; with <code>jobs.length > 0</code> and <code>limit >= 1</code> , <code>inflight</code> is non-empty here
21	loop exits when all started OR shutdown flagged; jobs already started keep running
27	grace timer created; NEVER cleared; if <code>Promise.all</code> wins the race, the timer still fires later: clause 4 violated (a pending timer after resolve)
28	waits for all in-flight or the grace period
30	marks unfinished as <code>ShutdownError</code> ; but those jobs are STILL RUNNING and will later write <code>results[i]</code> via line 15/16, overwriting the <code>ShutdownError</code> after <code>run</code> has resolved (harmless to the caller who already has the array reference? No: the caller holds the same array; it will change under them)
33	normal path: wait for the tail
36	listener removed on both paths; but if a job's <code>.then</code> handler threw (it cannot; it only assigns), or if <code>Promise.race</code> hung (23), line 36 is never reached and the listener leaks
4	<code>limit</code> not validated

Clauses

Clause	Status
1 concurrency, input order	yes (results indexed by <code>i</code>)
2 SIGTERM: stop, grace, resolve/reject rest	mostly; late writes overwrite <code>ShutdownError</code> (30 vs 15–16)
3 per-job failure isolation	yes (16)
4 no timers/listeners after resolve	NO: grace timer not cleared (27); listener leaks on hang (23)

Findings

- Line 27: grace timer never cleared; store the id and `clearTimeout` after the race; otherwise clause 4 fails on every shutdown where jobs finish early.
- Lines 30 vs 15–16: jobs still running after shutdown later overwrite the `ShutdownError` entries in the array the caller already holds; either detach them (ignore late results by checking `shuttingDown` in 15/16) or copy `results` before returning.
- Line 23: `Promise.race` on an empty set hangs forever; happens when `limit <= 0`; validate `limit >= 1`.
- Line 11/36: if anything above throws or hangs, the SIGTERM listener leaks; wrap in `try/finally`.

Positives: results by index preserve order; `.catch` per job isolates failures; `inflight` as a `Set` of the chains; listener removed on the happy path; `new Array(n)` sparseness used deliberately.

Confidence 2 · Request changes.

Clauses 1 and 3 are right; 2 and 4 fail on the shutdown path.

Program I — Idempotent payment endpoint

Requirement

1. `POST /pay` with `Idempotency-Key` header charges once per key; a repeat with the same key returns the original response (status and body) without charging again.
2. Keys expire after 24 h.
3. If the charge call throws, respond 502 and do NOT store the key, so the client may retry.
4. Concurrent requests with the same key: the second waits for the first and returns its response.
5. Amount must be a positive integer number of cents $\leq 1\,000\,000$.

```
1  type Stored = { status: number; body: unknown; at: number };
2  const store = new Map<string, Stored | Promise<Stored>>();
3  const TTL = 24 * 3600 * 1000;
4
5  app.post("/pay", async (req, res) => {
6    const key = req.headers["idempotency-key"];
7    if (typeof key !== "string" || key.length === 0) return res.status(400).json({ error: "key"
});
8    const amount = req.body.amount;
9    if (!Number.isInteger(amount) || amount <= 0 || amount > 1_000_000) return
res.status(400).json({ error: "amount" });
10
11    const existing = store.get(key);
12    if (existing) {
13      const r = await existing;
14      if (Date.now() - r.at < TTL) return res.status(r.status).json(r.body);
15      store.delete(key);
16    }
17
18    const work = (async (): Promise<Stored> => {
19      try {
20        const charge = await gateway.charge(amount);
21        return { status: 200, body: { id: charge.id }, at: Date.now() };
22      } catch (e) {
23        return { status: 502, body: { error: "gateway" }, at: Date.now() };
24      }
25    })();
26    store.set(key, work);
27    const r = await work;
28    store.set(key, r);
29    res.status(r.status).json(r.body);
30  });
```

Reading sequence

Step	Lines
types	1
state	2–3
signature	5
body	6–29
helpers	18–25 (inline async IIFE)
lifecycle	none (no sweep!)
clauses	

Notes

Line	Note
2	values are either a finished record or an in-flight Promise: the clause-4 mechanism
7	key validated as non-empty string: good (<code>string[]</code> and <code>undefined</code> rejected)
9	amount validated: integer, > 0 , $\leq 1e6$: clause 5 satisfied; <code>NaN</code> rejected by <code>isInteger</code>

Line	Note
11–12	<code>existing</code> is a record OR a Promise; truthy either way
13	<code>await existing</code> : on a record, <code>await</code> is harmless; on a Promise, waits: clause 4
14	TTL checked; returns stored response: clause 1
15	expired: delete and fall through to recharge
18–25	the IIFE NEVER rejects: the <code>catch</code> converts a gateway failure into a 502 RECORD
26	Promise stored before the await: concurrent second request at line 11 finds it: correct ordering
27–28	replaces the Promise with the record
23 → 28	a 502 result is STORED under the key with a timestamp; the client's retry with the same key hits line 14 and gets the stored 502 forever (24 h) without a new charge attempt: clause 3 violated ("do NOT store the key" on failure)
2	no eviction: expired entries are deleted only when a request with that key arrives (15); keys never seen again stay forever; clause 2 is observed on read but memory grows
29	sends; no <code>return</code> but last statement

Clauses

Clause	Status
1 once per key, replay response	yes
2 24 h expiry	observed on read; no sweep (growth)
3 on failure, 502 and do not store	502 yes; NOT STORING: fails (23, 28)
4 concurrent same key share	yes (26, 13)
5 amount validation	yes

Findings

- Lines 22–23 with 28: a gateway failure is stored as a 502 record; retries replay the 502 instead of retrying the charge; on failure, `store.delete(key)` and respond 502 without storing (and make the IIFE reject so concurrent waiters also see the failure, or store a failure marker that is deleted immediately after responding).
- Line 2: no sweep; keys accumulate for the process lifetime; add a periodic eviction (and `unref` it).
- Line 13: `await existing` when `existing` is a Promise that resolves to a 502 record returns that 502 to the concurrent waiter too; correct behavior once finding 1 is fixed by deleting, but with the current code both callers get a cached failure.

Positives: key and amount validation are complete; Promise-in-map handles concurrency correctly; Promise stored before the first `await`; TTL honored on replay.

Confidence 2 • Request changes.

Four of five clauses hold; the failure-path clause is inverted, which is the one that costs money.

Program J — Tree flatten with cycle detection and path output

Requirement

1. `flatten(root)` returns every node's `{ id, path }` where `path` is the slash-joined ids from root to node.
2. Children may be missing or empty; a node may appear under several parents (DAG): output it once per path.
3. Cycles must throw `CycleError` naming the id where the cycle closes.
4. The input tree must not be mutated.
5. Depth may reach 10 000.

```
1  interface Node { id: string; children?: Node[] }
2  class CycleError extends Error { constructor(public id: string) { super(`cycle at ${id}`); }
}
3
4  export function flatten(root: Node): Array<{ id: string; path: string }> {
5    const out: Array<{ id: string; path: string }> = [];
6    const onPath = new Set<string>();
7    const stack: Array<{ node: Node; path: string[] }> = [{ node: root, path: [root.id] }];
8
9    while (stack.length) {
10     const { node, path } = stack.pop()!;
11     if (onPath.has(node.id)) throw new CycleError(node.id);
12     onPath.add(node.id);
13     out.push({ id: node.id, path: path.join("/") });
14     for (const c of node.children ?? []) {
15       path.push(c.id);
16       stack.push({ node: c, path });
17     }
18     onPath.delete(node.id);
19   }
20   return out;
21 }
```

Reading sequence

Step	Lines
types	1–2
state	5–7 (per call)
signature	4
body	9–20
helpers	none
lifecycle	none
clauses	

Notes

Line	Note
7	iterative stack: no recursion, so depth 10 000 is safe: clause 5
10	<code>pop()!</code> justified by the <code>while</code> guard
11–12, 18	<code>onPath</code> is meant to track ancestors on the CURRENT path; but with an explicit stack, line 18 runs right after pushing children, before any child is visited: <code>onPath</code> never contains an ancestor when a descendant is processed; cycle detection is dead; a cycle loops forever
13	pushes the record: fine
14	<code>?? []</code> handles missing children
15–16	<code>path.push(c.id)</code> MUTATES the shared <code>path</code> array and pushes the SAME array reference for every child: all siblings share one path array containing all sibling ids; the join at 13 for each child shows the wrong path; and the array keeps growing
14–17	DAG: a node under two parents is pushed twice with two paths: clause 2 would be right if the paths were right
1	input never mutated (no writes to <code>node</code>): clause 4 holds

Clauses

Clause	Status
1 id + slash path	path wrong for any node with siblings (15–16)
2 once per path	yes in principle
3 cycle → throw	NO: detection dead (18); infinite loop instead
4 no mutation of input	yes
5 depth 10 000	yes (iterative)

Findings

- Lines 15–16: `path` is shared and mutated; each child must get its own copy: `stack.push({ node: c, path: [...path, c.id] })`.
- Lines 11–12, 18: ancestor tracking does not work with an explicit stack; carry the ancestor set per stack entry (`new Set(ancestors)` plus the node), or store the path array and test `path.includes(c.id)` before pushing (O(depth) per child; acceptable), or use a DFS with explicit enter/exit markers.
- Line 7: `root` with a `children` cycle back to itself: `onPath` is empty at line 11 on the second visit, so it loops; same root cause as 2.

Positives: iterative traversal for depth; `?? []` for missing children; no mutation of input nodes; `pop()!` justified; `CycleError` carries the id.

Confidence 1 • Reject.

Two of the three functional clauses fail; one fails by hanging.

Scoring yourself

Program	Must-find	Verdict
H	grace timer not cleared (27); late writes overwrite ShutdownError (30); race on empty set (23)	2
I	502 stored and replayed (23 → 28); no sweep (2)	2
J	shared mutated <code>path</code> (15–16); dead cycle detection (18)	1

Each of these programs has more correct careful code than wrong code. If your notes column is mostly “fine, because...”, you read it right; the findings are the three or four rows that are not.